Files and Their Organization

Lecture 1: Fundamentals and File Organization Methods

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Why Do We Need Files?

Problem

Variables and dynamically allocated memory generally do not preserve application data after the program terminates.

- Applications need persistent storage.
- Data may contain thousands or millions of records.
- Records must be inserted, searched, modified, and deleted.
- Efficient organization affects access performance.

Example

A university must preserve student records across multiple academic sessions.

What Is a File?

Definition

A file is a collection of related records stored under a common name.

- Files provide persistent data storage.
- Files can store text, binary records, images, and other information.
- Files are identified through their names and locations.

Example

A student file may contain 60 student records, each having roll number, name, address, and marks.

Data Hierarchy

$$\text{Field} \longrightarrow \text{Record} \longrightarrow \text{File} \longrightarrow \text{Directory}$$

Field	Stores a single fact.
Record	Collection of related fields.
File	Collection of related records.
Directory	Organizes related files.

Student Example

`Roll Number` is a field.
`{Roll Number, Name, Marks}` is a record.
Multiple student records form a student file.

Data Field

Definition

A data field is an elementary unit that stores a single fact.

A field is characterized by:

- Data type
- Size
- Meaning

Field	Type	Example
Roll Number	Integer	1021
Name	Character array	Aman
Marks	Floating point	86.5

Record

Definition

A record is a collection of related data fields treated as one logical unit.

Field	Value
Roll Number	1021
Name	Aman
Course	B.Tech CSE
Marks	86.5

- Records describe individual entities.
- A key field may uniquely identify a record.

Representing a Record in C

```c
#include <stdio.h>

struct Student {
    int rollNo;
    char name[30];
    float marks;
};

int main(void) {
    struct Student s = {1021, "Aman", 86.5f};

    printf("Roll: %d\n", s.rollNo);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

Observation

A C structure is a natural representation of a record containing related fields.

Directory Organization

A directory stores information about related files.

Student Directory
Personal Information File
Academic Information File
Fees Information File

- Personal file: roll number, name, address.
- Academic file: course, marks, grades.
- Fees file: fees, hostel dues, library dues.

Purpose

Directories help organize and locate files.

Introduction to File Attributes

Each file has associated attributes or metadata.

- File name
- File position
- File structure
- File access method
- Attribute flags

Purpose

Attributes describe how a file is identified, accessed, and managed.

File Name and File Position

File Name

A character string used to identify a file.

Examples:

- `students.txt`
- `marks.dat`
- `attendance.bin`

File Position

The current location at which the next read or write operation will occur.

- Position changes as data is read or written.
- It may be changed explicitly using file positioning functions.

File Access Methods

Sequential Access

- Records are accessed in sequence.
- Earlier records are traversed to reach a later record.
- Useful for batch processing.

Random Access

- Records may be accessed directly.
- Position can be calculated or indexed.
- Useful for interactive applications.

Example

To read student record 39, sequential access may require examining the first 38 records, whereas random access can seek directly to its location.

Common File Attribute Flags

Attribute	Purpose
Read-only	Restricts modification
Hidden	Hides normal directory listing
System	Identifies system-related file
Volume Label	Identifies storage volume
Directory	Identifies directory entry
Archive	Indicates modification for backup

Note

The six-bit attribute representation described in the textbook is associated with traditional DOS-style file attributes. Modern file systems may use different metadata and permission models.

Attribute Flag Representation

Attribute	Bit Pattern
Read-only	00000001
Hidden	00000010
System	00000100
Volume Label	00001000
Directory	00010000
Archive	00100000

Worked Example

A file that is read-only, hidden, and system-marked:

$$00000001 \mid 00000010 \mid 00000100 = \boxed{00000111}$$

Archive Attribute

The archive bit helps identify files modified since a previous backup.

1. A file is modified.
2. Its archive bit is set.
3. A backup program checks the bit.
4. The modified file is backed up.
5. The backup program may clear the bit.

Application

Incremental backup systems.

Text Files

Definition

A text file stores data as a sequence of characters, commonly arranged into lines.

- Human-readable using a text editor.
- Numbers are represented as characters.
- Suitable for logs, configuration, and simple data exchange.

Example

The number 123 may be stored as the characters `'1'`, `'2'`, and `'3'`.

Binary Files

Definition

A binary file stores data as bytes in an application-defined binary representation.

- Not necessarily human-readable.
- May store fixed-size records.
- Can support efficient direct record access.
- Interpretation depends on the file format.

Important

Raw C structures written as binary data may contain padding and platform-dependent representations. Such files are not automatically portable.

Text Files vs Binary Files

Feature	Text File	Binary File
Representation	Character encoding	Binary byte format
Readability	Often human-readable	Usually needs a program
Numeric values	Stored as character sequences	Stored according to binary format
Editing	Text editors	Specialized software
Examples	CSV, TXT	DAT, binary record file

C Program: Writing a Text File

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("students.txt", "w");

    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    fprintf(fp, "%d %s %.2f\n",
            101, "Aman", 85.50);
    fprintf(fp, "%d %s %.2f\n",
            102, "Riya", 91.25);

    fclose(fp);
    return 0;
}
```

Output File

```
101 Aman 85.50
102 Riya 91.25
```

C Program: Reading a Text File

```c
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("students.txt", "r");
    int roll;
    char name[30];
    float marks;

    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    while (fscanf(fp, "%d %29s %f",
                  &roll, name, &marks) == 3) {
        printf("%d %s %.2f\n",
               roll, name, marks);
    }

    fclose(fp);
    return 0;
}
```

C Program: Writing Binary Records

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void) {
    struct Student s = {101, "Aman", 85.5f};

    FILE *fp = fopen("student.dat", "wb");
    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    if (fwrite(&s, sizeof(s), 1, fp) != 1) {
        perror("Write failed");
        fclose(fp);
        return 1;
    }

    fclose(fp);
    return 0;
}
```

C Program: Reading Binary Records

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void) {
    struct Student s;
    FILE *fp = fopen("student.dat", "rb");

    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    if (fread(&s, sizeof(s), 1, fp) == 1) {
        printf("%d %s %.2f\n",
               s.roll, s.name, s.marks);
    }

    fclose(fp);
    return 0;
}
```

Basic File Operations

The textbook groups file operations into:

1. **Creation**
2. **Updation**
   - Insertion
   - Modification
   - Deletion
3. **Retrieval**
   - Inquiry
   - Report generation
4. **Maintenance**
   - Restructuring
   - Reorganization

Creating a File

1. Specify the file name.
2. Select the appropriate opening mode.
3. Open the file.
4. Write records.
5. Close the file.

C Functions

`fopen()`, `fprintf()`, `fwrite()`, `fclose()`

Note

Opening an existing file with mode `"w"` or `"wb"` truncates its contents.

Updating a File

Updating means changing file contents to reflect current information.

Insertion Add a new student record.

Modification Correct a student's name.

Deletion Remove a withdrawn student's record.

Example

If a student changes their address, the corresponding record must be updated.

Important

The implementation of updates depends on the chosen file organization.

Retrieval and Maintenance

Retrieval

Extracting information from a file.

- Inquiry: retrieve a particular student.
- Report: generate marks for an entire class.

Maintenance

Improving the organization or structure of a file.

- Restructuring: change record fields or widths.
- Reorganization: change the organization method.

What Is File Organization?

Definition

File organization is the logical arrangement of records within a file.

- Determines how records are stored and accessed.
- Affects search and update performance.
- Influences storage utilization.
- Depends on application requirements.

Design Question

Should student records be processed in order, accessed directly by roll number, or located through an index?

Factors Affecting File Organization

Before selecting a method, consider:

1. Speed of record retrieval.
2. Ease of insertion.
3. Ease of modification.
4. Ease of deletion.
5. Efficient storage utilization.
6. Data integrity requirements.
7. Sequential versus random access.

Key Idea

There is no single organization method that is best for every application.

Major File Organization Methods

Method	**Main Principle**
Sequential	Records processed in sequence
Relative	Direct access using record number
Indexed Sequential	Index maps keys to record locations

These three methods form the core of the present lecture.

Sequential File Organization

Definition

Records are maintained and accessed in sequence.

Record 0	Record 1	Record 2	Record 3	Record 4

- Records are read one after another.
- New records may be appended.
- Commonly used in batch processing.
- Suitable for sequential reports.

Sequential File: Worked Example

Consider student records:

Position	Roll Number
1	101
2	102
3	103
4	104
5	105

Task

Locate roll number 104 using sequential scanning.

Comparisons:

$$101 \rightarrow 102 \rightarrow 103 \rightarrow 104$$

4 records examined

Sequential File: Search Procedure

1. Open the file.
2. Read the first record.
3. Compare its key with the target.
4. If matched, return the record.
5. Otherwise, read the next record.
6. Continue until found or end of file.

Complexity

For an unsorted sequential scan of n records:

$$T_{\text{best}} = O(1)$$

$$T_{\text{average}} = O(n)$$

$$T_{\text{worst}} = O(n)$$

C Program: Sequential Search in a File

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
};

int main(void) {
    FILE *fp = fopen("students.dat", "rb");
    struct Student s;
    int target = 104, found = 0;

    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    while (fread(&s, sizeof(s), 1, fp) == 1) {
        if (s.roll == target) {
            printf("Found: %s\n", s.name);
            found = 1;
            break;
        }
    }

    if (!found)
        puts("Record not found");

    fclose(fp);
    return 0;
}
```

Sequential File: Advantages

- Simple organization.
- Easy sequential processing.
- Minimal indexing overhead.
- Suitable for batch-oriented applications.
- Can be implemented on sequential storage devices.

Applications

- Payroll processing
- Sequential report generation
- Processing transaction logs

Sequential File: Limitations

- Random record retrieval is inefficient.
- Finding later records requires scanning.
- Maintaining sorted order can be costly.
- Updates may require rewriting a file.
- Not ideal for frequent interactive queries.

Textbook Context

The textbook describes a restrictive sequential organization in which updates and deletions require creating a replacement file.
Ordinary sequential files may still be appended to or modified using suitable file operations.

Relative File Organization

Definition

Records are located using a relative record number.

- Records have fixed length.
- Record numbers identify logical slots.
- Some slots may be marked FREE.
- Supports direct and sequential access.

0	1	2	3	4
R0	R1	FREE	FREE	R4

Relative File: Address Calculation

For zero-based record numbers:

$$\boxed{\text{Offset}(r) = r \times L}$$

where:

- r is the relative record number.
- L is the fixed record length.

If a conceptual base address B is given:

$$\boxed{\text{Address}(r) = B + rL}$$

For the ith record using one-based ordinal numbering:

$$\boxed{\text{Address}(i) = B + (i - 1)L}$$

Relative File: Numerical Example

Given

- Base address $B = 1000$
- Record length $L = 20$ bytes
- Find the address of the 5th record.

Using one-based ordinal position:

$$\text{Address}(5) = 1000 + (5 - 1) \times 20$$

$$= 1000 + 80$$

$$\boxed{1080}$$

Remember

The 5th record has zero-based record number 4.

Relative File: Second Example

A file contains fixed-size records of 64 bytes.
Find the byte offset of relative record number 25.

$$\text{Offset}(r) = r \times L$$

$$= 25 \times 64$$

$$\boxed{1600 \text{ bytes}}$$

Interpretation

Record number 25 begins 1600 bytes from the beginning of the file.

C Program: Direct Access Using fseek()

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[30];
    float marks;
};

int main(void) {
    FILE *fp = fopen("students.dat", "rb");
    struct Student s;
    long recordNo = 4;

    if (fp == NULL) {
        perror("Cannot open file");
        return 1;
    }

    if (fseek(fp,
              recordNo * (long)sizeof(s),
              SEEK_SET) != 0) {
        perror("Seek failed");
        fclose(fp);
        return 1;
    }

    if (fread(&s, sizeof(s), 1, fp) == 1)
        printf("%d %s %.2f\n",
               s.roll, s.name, s.marks);

    fclose(fp);
```

Relative File: Advantages

- Fast direct record access.
- Supports sequential access.
- Updates can be performed in place.
- Records can occupy designated slots.
- Suitable for interactive applications.

Application

A student record system where each student has a known relative record number.

Relative File: Disadvantages

- Requires fixed-size records.
- Relative record number must be known.
- Unused slots can waste storage.
- Primarily suited to random-access devices.
- Large gaps between record numbers may increase file size.

Example

If only record numbers 0 and 999 are used, many intermediate slots may remain unused.

Indexed Sequential File Organization

Definition

Indexed sequential organization maintains an index that helps locate records efficiently.

- Records are identified by search keys.
- Index entries provide record locations.
- Supports sequential and direct retrieval.
- Requires additional index storage.

Index Table: Example

Record Number	Record Address
1	120
2	350
3	510
4	742
5	876

Task

Find the fourth record.

1. Locate record number 4 in the index.
2. Read its stored address: 742.
3. Access the record at address 742.

Indexed Sequential Search: Steps

1. Identify the required search key.
2. Search the index for the relevant entry.
3. Obtain the record or block address.
4. Access the corresponding location.
5. Retrieve the desired record.

Key Benefit

The index can reduce the number of data records that must be examined.

Indexed Sequential: Practical Example

A college maintains student information.

Sequential Requirement

Generate a report of marks for all students.

Random Access Requirement

Locate one student and correct their name.

Why an Index Helps

The index supports targeted record retrieval, while ordered data access supports report generation.

Indexed Sequential: Advantages

- Faster targeted retrieval than full scans.
- Supports batch and interactive processing.
- Provides sequential and random access.
- Records can be updated within the file.
- Suitable for large collections of records.

Indexed Sequential: Disadvantages

- Requires extra space for index entries.
- Index maintenance adds overhead.
- More complicated than sequential files.
- Index updates may be needed after insertions and deletions.
- Typically relies on random-access storage.

Comparison of File Organization Methods

Feature	Sequential	Relative	Indexed Sequential
Access	Sequential	Direct and sequential	Indexed and sequential
Record size	Often fixed in textbook model	Fixed	Fixed in textbook model
Random retrieval	Slow	Fast by record number	Fast using index
Extra index	Not required	Not required	Required
Updates	May require rewrite	Can be in place	Index maintenance needed
Typical use	Batch reports	Direct record access	Mixed workloads

Choosing the Right Organization

Scenario 1

Process all employee salary records every month.

Suitable choice: Sequential organization.

Scenario 2

Access fixed-size records using known slot numbers.

Suitable choice: Relative organization.

Scenario 3

Generate reports and perform individual record lookups.

Suitable choice: Indexed sequential organization.

Classroom Activity 1

Problem

A relative file has:

- Base address = 5000
- Record size = 48 bytes

Find the address of the 12th record.

Students should:

1. Identify the record position.
2. Apply the correct formula.
3. Show the final address.

Classroom Activity 1: Solution

For the 12th record:

$$B = 5000, \quad L = 48, \quad i = 12$$

$$\text{Address}(i) = B + (i - 1)L$$

$$= 5000 + (12 - 1) \times 48$$

$$= 5000 + 528$$

$$\boxed{5528}$$

Classroom Activity 2

Problem

A file contains 1000 unsorted records.
A sequential search looks for a record that is located at position 800.

1. How many records are examined?
2. What is the worst-case time complexity?
3. Which organization may support faster direct access if the relative record number is known?

Classroom Activity 2: Solution

1. Records examined:

$$\boxed{800}$$

2. Worst-case complexity:

$$\boxed{O(n)}$$

3. Suitable organization:

$$\boxed{\text{Relative file organization}}$$

Reason

Relative organization can calculate the location of a fixed-length record directly.

MCQ 1

Which of the following represents the correct data hierarchy?

1. File → Field → Record
2. Field → Record → File
3. Record → File → Field
4. Directory → Field → Record

Correct Answer

Option 2: Field → Record → File

MCQ 2

Which file organization provides direct access using a relative record number?

1. Sequential organization
2. Relative organization
3. Unordered sequential scan
4. None of the above

Correct Answer

Option 2: Relative organization

MCQ 3

What is the main purpose of an index table?

1. To delete all records automatically
2. To store only file names
3. To locate records efficiently
4. To convert binary files into text

Correct Answer

Option 3: To locate records efficiently

MCQ 4

Which C function changes the file position?

1. `printf()`
2. `malloc()`
3. `fseek()`
4. `strlen()`

Correct Answer

Option 3: `fseek()`

MCQ 5

For a zero-based relative record number r and record size L, what is the byte offset?

1. $r + L$
2. rL
3. $(r + 1)L$
4. r/L

Correct Answer

Option 2: rL

Lecture Summary

- A file is a collection of related records.
- Records consist of related data fields.
- File attributes describe file properties.
- Text and binary files differ in representation.
- File operations include creation, updating, retrieval, and maintenance.
- Sequential organization is useful for ordered processing.
- Relative organization supports direct access.
- Indexed sequential organization combines indexing with ordered access.

Questions for Revision

1. Differentiate between a field and a record.
2. Explain any four file attributes.
3. Compare text and binary files.
4. What is sequential file organization?
5. Derive the address formula for a relative file.
6. Explain indexed sequential file organization.
7. Compare all three file organization methods.
8. Write a C program to access a fixed-size binary record using `fseek()`.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 16: Files and Their Organization.
2. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*.
3. Standard C file I/O functions: `fopen()`, `fclose()`, `fread()`, `fwrite()`, and `fseek()`.

Student Feedback

Share Your Feedback



Thank you!